CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Section 2: Multi-Agent Orchestration & Subagent Design

Exam objectives covered

1. Construct subagent prompts that include all findings, structured data, and source metadata required for task completion without returning to the coordinator for missing context.
2. Evaluate subagent delegation strategies—goal-oriented versus procedural instructions—and select the approach that enables adaptive behavior while maintaining coordinator visibility and control.
3. Configure subagent invocations with appropriate tool restrictions, context scoping, and system prompts that constrain each agent to its designated role.
4. Diagnose misconfigured subagent spawning by identifying missing tool permissions, incorrect AgentDefinition parameters, or absent coordinator-to-subagent wiring.
5. Decompose complex tasks into dynamically generated subtasks that adapt as new information is discovered, rather than executing a fixed sequence regardless of intermediate findings.
6. Select the appropriate agentic review architecture—plan mode, direct execution, or multi-phase workflow—based on task scope, risk level, and human approval requirements.
7. Design state persistence strategies for multi-agent pipelines that enable reliable resumption after interruption without repeating completed work or losing prior findings.
8. Evaluate multi-agent orchestration patterns—coordinator-worker, parallel execution, and sequential pipelines—to select the structure best satisfying research coverage, latency, and reliability requirements.
9. Design subagent output schemas that render structured data, prose summaries, and citation metadata in the format best suited to downstream synthesis and reporting.
10. Design synthesis agent behavior that preserves source-level uncertainty, distinguishing well-established findings from contested claims rather than collapsing conflicting data into single confident statements.
11. Configure tool distribution in multi-agent systems by assigning each subagent only the tools required for its designated role, reducing decision complexity and preventing out-of-role tool invocations.

Q1.

A coordinator delegates a document-analysis task to a subagent. Which prompt design best aligns with effective subagent context packaging?

A. Give the subagent only the task goal and let it request missing files later B. Include the goal, relevant findings so far, structured input data, file/source metadata, and output requirements C. Provide a general objective and trust the subagent to search all repositories for context D. Ask the subagent to infer missing schema details from previous runs

Answer: B **Explanation:** A strong subagent prompt should be self-sufficient: it includes the task, prior findings, relevant structured inputs, source metadata, and the required output format. A, C, and D all increase rework or ambiguity by forcing the subagent to recover missing context or make unsafe assumptions.

Q2.

You want a research subagent to adapt its investigation if it uncovers new evidence. Which delegation style is most appropriate?

A. A fixed procedural checklist with no deviations allowed B. Goal-oriented instructions with clear constraints, success criteria, and reporting requirements C. An unbounded prompt with no role limits so the subagent can be creative D. A short instruction to "figure it out however you want"

Answer: B **Explanation:** Goal-oriented prompting supports adaptive behavior while preserving coordinator control through explicit constraints and expected deliverables. A is too rigid, while C and D reduce predictability, increase role drift, and make it harder to audit the subagent's reasoning path.

Q3.

A compliance-review subagent only needs read-only file access and a search tool. What is the best tool configuration?

A. Grant all available tools to avoid accidental blocking B. Grant read-only file access, search, and shell execution just in case C. Grant only the search and read-only file tools required for the assigned role D. Grant no tools and require the subagent to reason from memory

Answer: C **Explanation:** Least-privilege tool assignment reduces decision complexity and prevents out-of-role behavior. A and B over-provision capabilities, increasing risk, while D undermines task quality because the subagent cannot inspect the needed artifacts.

Q4.

A subagent never starts, even though the coordinator appears to call it. Which diagnosis most directly fits a spawning misconfiguration?

A. The output schema included both JSON and prose fields B. The AgentDefinition or invocation parameters are incorrect, so the coordinator is not actually wiring the subagent call properly C. The coordinator used a goal-oriented prompt instead of a procedural one D. The subagent returned too much evidence in its report

Answer: B **Explanation:** If a subagent never launches, missing coordinator-to-subagent wiring or incorrect invocation parameters are prime suspects. A and D are downstream output issues, and C affects behavior quality, not whether the subagent is instantiated at all.

Q5.

Which example best reflects dynamic task decomposition?

A. Splitting work into five predetermined subtasks before any evidence is reviewed B. Executing the same fixed pipeline for every incident regardless of findings C. Creating initial subtasks, then generating new ones when new entities, dependencies, or risks are discovered D. Asking one subagent to do everything to avoid orchestration overhead

Answer: C **Explanation:** Dynamic decomposition means the workflow evolves as new information appears. A and B are fixed plans, while D avoids decomposition entirely and sacrifices scalability, specialization, and transparency.

Q6.

A team must assess a high-risk deployment change, requires human approval before remediation, and wants traceable intermediate outputs. Which architecture is best?

A. Direct single-agent execution with no checkpoints B. Plan mode only, with no execution support C. Multi-phase workflow with explicit review/approval boundaries between phases D. Parallel execution of all steps with automatic commit at the end

Answer: C **Explanation:** High-risk work with approval requirements benefits from multi-phase orchestration because it preserves traceability and introduces controlled decision points. A and D reduce oversight, while B may help planning but does not by itself support the full controlled execution lifecycle.

Q7.

What is the most reliable persistence strategy for a multi-agent pipeline that may be interrupted and resumed later?

A. Store everything only in the coordinator's short-term memory B. Persist completed task states, structured outputs, and checkpoints in durable storage with resumable status markers C. Re-run the entire pipeline from the beginning after any failure D. Save only final prose summaries and discard intermediate data

Answer: B **Explanation:** Durable state with resumable checkpoints prevents repeated work and preserves prior findings. A is fragile, C is wasteful and error-prone, and D loses the structured intermediate artifacts needed for reliable continuation.

Q8.

Which orchestration pattern is usually best when you need broad research coverage quickly across independent sources?

A. Sequential pipeline B. Coordinator-worker with parallel workers C. Single-agent direct execution D. Manual human review only

Answer: B **Explanation:** Parallel workers coordinated centrally improve coverage and latency when tasks are independent. A can be too slow, C limits specialization and concurrency, and D may be

useful for approval but not for scalable source coverage.

Q9.

A synthesis agent will combine outputs from several research subagents. Which output schema from the workers is most useful downstream?

A. Unstructured free-form essays only B. A single confidence score with no citations C. Structured findings, concise narrative summary, and source/citation metadata for each claim D. Raw copied source text with no interpretation

Answer: C **Explanation:** Downstream synthesis works best when each worker returns machine-usable structure plus human-readable summary and traceable citations. A and D make aggregation harder, while B is too thin to support verification or nuanced synthesis.

Q10.

Two subagents return conflicting conclusions about a vendor's support status. How should the synthesis agent respond?

A. Pick the answer from the faster subagent to reduce latency B. Merge the results into one confident statement to avoid confusing the user C. Preserve the disagreement, attribute claims to their sources, and indicate uncertainty or need for further verification D. Discard both results because inconsistency means they are unusable

Answer: C **Explanation:** Good synthesis preserves source-level uncertainty instead of flattening conflict into false certainty. A and B hide important reliability signals, and D is overly dismissive because conflicting data often still contains valuable evidence.

Q11.

Which tool-distribution choice best follows least-privilege design in a multi-agent system?

A. Every subagent gets the same complete tool set for consistency B. Only the coordinator gets tools; subagents get none C. Each subagent receives only the minimal tools needed for its designated role D. Tools are granted dynamically without regard to agent role, as long as performance is fast

Answer: C **Explanation:** Role-specific minimal tooling reduces risk and keeps behavior aligned with design intent. A and D increase misuse potential, while B can make subagents ineffective when they legitimately need limited access to perform their jobs.

Q12.

A coordinator asks a subagent to "review these findings" but forgets to include the actual findings and source links. What is the most likely outcome?

A. Better reasoning because the subagent is not biased by prior evidence B. Efficient completion because the subagent can reconstruct everything cheaply C. Delays, incomplete analysis, or a need to re-query the coordinator for missing context D. No effect, because source metadata is optional in all cases

Answer: C **Explanation:** Missing findings and source metadata usually force recovery work, increase ambiguity, and break self-contained delegation. A and B are unrealistic, and D is wrong because source context is often essential for verification and accurate downstream synthesis.

Q13.

Which coordinator instruction best preserves visibility and control while still allowing adaptive subagent behavior?

A. "Follow these 27 steps exactly; do not deviate under any circumstances." B. "Find the root cause, stay within these tool limits, cite every claim, and report any blockers or branching decisions." C. "Use any tools you want and only return the final answer." D. "Try something reasonable and skip details unless asked."

Answer: B **Explanation:** This instruction is goal-oriented but bounded: it defines the objective, constraints, reporting expectations, and escalation path. A over-constrains adaptation, while C and D reduce auditability, control, and reliability.

Q14.

A subagent intended to query a database keeps failing with permission errors. Which issue is the best first hypothesis?

A. The subagent was given too much context B. The coordinator used citations in the prompt C. The subagent invocation is missing the required database tool permission or capability binding D. The subagent needs a longer prose explanation of the task

Answer: C **Explanation:** Permission failures usually indicate missing tool access or misconfigured capabilities. A, B, and D may affect clarity or efficiency, but they do not directly explain authorization or tool-availability errors.

Q15.

When is direct execution generally more appropriate than multi-phase orchestration?

A. Large, high-risk changes requiring audit trails and approvals B. Small, low-risk tasks with limited scope and minimal coordination overhead C. Research tasks across many independent data sources with conflicting evidence D. Pipelines that must resume after interruption across many stages

Answer: B **Explanation:** Direct execution is a good fit when the task is simple and low risk, because orchestration overhead would add little value. A, C, and D all benefit more from richer agentic structure, checkpoints, or specialization.

Q16.

Which subagent system prompt is best designed to constrain role while supporting high-quality outputs?

A. "You are a general assistant. Help however you think best." B. "You are a citation-focused evidence extractor. Use only search and read tools, do not make final policy decisions, and return

findings in the specified schema." C. "You may use any method as long as you finish quickly." D. "Act like the coordinator and decide whether remediation should proceed."

Answer: B **Explanation:** This prompt clearly defines role boundaries, tool limits, and output expectations. A and C are too open-ended, and D collapses separation of concerns by having the subagent take over the coordinator's decision role.

Q17.

A team wants fast answers and also wants failures in one branch not to invalidate already completed work in others. Which pattern best fits?

A. Parallel workers with independent state persistence per branch and coordinator synthesis B. One sequential chain where each stage depends on the previous one completing C. A single monolithic agent run with no checkpointing D. A fixed procedural loop that retries the full workflow after any error

Answer: A **Explanation:** Parallel branches with persisted outputs maximize latency benefits and isolate failures so completed branches remain usable. B and D create unnecessary coupling and rework, while C reduces resilience and transparency.

Q18.

Which worker output is least suitable for downstream synthesis in a multi-agent pipeline?

A. `{finding, evidence, source_id, confidence, caveats}` plus a short summary B. A table of extracted entities with provenance and notes C. A prose paragraph that mixes conclusions, evidence, and assumptions without source attribution D. A JSON object with citations and unresolved questions

Answer: C **Explanation:** Unattributed mixed prose is hardest to validate, merge, and audit. A, B, and D all preserve structured information or provenance that helps a synthesis agent reason about reliability and combine results safely.

Q19.

A coordinator must choose between a procedural subagent prompt and a goal-oriented one. Which factor most strongly favors the goal-oriented approach?

A. The environment is stable and every case follows the same exact path B. The task may branch based on intermediate discoveries, but the coordinator still needs bounded outputs and observability C. The subagent should imitate human brainstorming with no constraints D. The coordinator wants to hide task details from the subagent

Answer: B **Explanation:** Goal-oriented prompting is strongest when the task may evolve based on findings, yet still needs guardrails and visible deliverables. A leans more toward procedural flow, while C and D are incompatible with disciplined orchestration.

Q20.

In a resumable multi-agent investigation, which persisted state design is most complete?

A. Only store the final answer after all agents finish B. Store agent names and timestamps only C. Store task status, assigned agent role, input snapshot, tool scope, outputs, citations, and next-action/resume markers D. Store raw logs only and reconstruct state manually later

Answer: C **Explanation:** Reliable resumption requires enough structured state to know what ran, under what constraints, what it produced, and what remains. A and B omit critical execution context, while D places too much burden on manual reconstruction and increases recovery error.